LAB MANUAL

SUBJECT: OE (WEB DESIGNING)

Sr No	Title of Experiments	Technology
1	Static Web Page Development	HTML + CSS
2	Interactive Forms with JavaScript Validation	JavaScript
3	JavaScript DOM Manipulation and Event Handling	
4	JavaScript Control Flow and Simple Application	
5	PHP Variable Handling and Data Types	PHP
6	PHP Conditional Logic and Looping	
7	Session and Cookie Management in PHP	
8	Database Connection with PHP	PHP + MySQL
9	Display Data from Database using PHP	PHP
10	Mini Project – Registration System	HTML + CSS + JavaScript + PHP + MySQL

Experiment 1: Static Web Page Development using HTML and CSS

Objective: To design and create a simple static website using HTML and CSS.

Theory:

HTML (HyperText Markup Language)

- **Purpose:** HTML is the **standard markup language** used to create and structure content on the web.
- **Role:** Think of it as the **skeleton** of a web page—it defines what elements exist and in what order.
- **Features:**
    - **Headings & Paragraphs:** Structure text into readable sections.
    - **Lists (ordered/unordered):** Present information clearly.
    - **Links (Hyperlinks):** Connect web pages and external resources.
    - **Tables:** Organize tabular data.
    - **Forms:** Collect user input such as text, email, or passwords.
    - **Multimedia (images, audio, video):** Enrich content presentation.
- **Limitation:** HTML focuses only on **structure and meaning** of the content, not the design or style.

Example (HTML only):

```
<h1>My Website</h1>
<p>Welcome to my first web page.</p>
<ul>
  <li>Home</li>
  <li>About</li>
  <li>Contact</li>
</ul>
```

CSS (Cascading Style Sheets)

- **Purpose:** CSS controls the **look and feel** of web pages.
- **Role:** Acts like the **skin and clothing** of the skeleton provided by HTML.
- **Features:**
    - **Typography:** Change font style, size, and color.
    - **Colors & Backgrounds:** Add visual appeal to elements.
    - **Spacing & Borders:** Control padding, margins, and decorative borders.
    - **Layout:** Arrange content using techniques like Flexbox and Grid.
    - **Positioning:** Place elements (relative, absolute, fixed, sticky).
    - **Responsive Design:** Ensure pages adapt to different screen sizes.
- **Advantage:** By separating content (HTML) from style (CSS), developers can **maintain code easily** and **reuse styling rules** across multiple pages.

Example (Adding CSS):

```
h1 {
  color: navy;
  text-align: center;
  font-family: Arial, sans-serif;
}
p {
  font-size: 16px;
  color: gray;
}
ul {
  list-style-type: square;
  background: #f2f2f2;
  padding: 10px;
}
```

HTML + CSS Together

- HTML provides the **content & structure**, CSS provides the **style & design**.
- Together they ensure a page is both:
    - **Meaningful** (accessible and structured).
    - **Attractive** (visually engaging and user-friendly).

Procedure:

1. Create a folder named exp1.
2. Inside it, create a file index.html.
3. Write HTML code to add headings, paragraphs, and lists.
4. Create a style.css file for CSS styling.
5. Link the CSS file inside the <head> section of index.html.
6. Save the files and open index.html in a web browser.

Code:

Index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>My Static Website</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <header>
    <h1>Welcome to My Website</h1>
    <nav>
      <ul>
        <li><a href="index.html">Home</a></li>
        <li><a href="about.html">About</a></li>
        <li><a href="contact.html">Contact</a></li>
      </ul>
    </nav>
  </header>

  <section>
    <h2>Introduction</h2>
    <p>This is a simple static website created using HTML and CSS.</p>
    <img src="travel.jpg" alt="Sample Image">
  </section>
  <section>
    <h2>Features</h2>
    <ul>
      <li>Easy to use</li>
      <li>Lightweight</li>

    </ul>
  </section>

  <footer>
    <p>&copy; 2025 My Static Website</p>
  </footer>
</body>
</html>
```

about.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>About - My Static Website</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <header>
    <h1>About Us</h1>
    <nav>
      <ul>
        <li><a href="index.html">Home</a></li>
        <li><a href="about.html">About</a></li>
        <li><a href="contact.html">Contact</a></li>
      </ul>
    </nav>
  </header>

  <section>
    <h2>Who We Are</h2>
    <p>We are a team passionate about building simple and beautiful websites.</p>
  </section>
  <section>
    <h2>Team Members</h2>
    <table>
      <tr>
        <th>Name</th>
        <th>Role</th>
      </tr>
      <tr>
        <td>Alice</td>
        <td>Designer</td>
      </tr>
      <tr>
        <td>Bob</td>
        <td>Developer</td>
      </tr>
      <tr>
        <td>Charlie</td>
```

```
<td>Tester</td>
        </tr>
      </table>
  </section>

  <footer>
    <p>&copy; 2025 My Static Website</p>
  </footer>
</body>
</html>
```

contact.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Contact - My Static Website</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <header>
    <h1>Contact Us</h1>
    <nav>
      <ul>
        <li><a href="index.html">Home</a></li>
        <li><a href="about.html">About</a></li>
        <li><a href="contact.html">Contact</a></li>
      </ul>
    </nav>
  </header>
  <section>
    <h2>Get in Touch</h2>
    <p>You can reach us through the following platforms:</p>
    <ul>
      <li>Email: <a href="mailto:info@example.com">info@example.com</a></li>
      <li>Phone: +91 9876543210</li>
      <li>Website: <a href="https://www.example.com"   target="_blank">www.example.com</a>
          </li>
    </ul>
  </section>
```

```
  <footer>
    <p>&copy; 2025 My Static Website</p>
  </footer>
</body>
</html>
```

Style.css

```
/* General Reset */
body {
  margin: 0;
  font-family: Arial, sans-serif;
  background-color: #f4f4f9;
  color: #333;
}

/* Header */
header {
  background-color: #333;
  color: #fff;
  padding: 15px 20px;
  text-align: center;
}

header h1 {
  margin: 0;
}

nav ul {
  list-style: none;
  padding: 0;
  margin: 10px 0 0;
  display: flex;
  justify-content: center;
  gap: 20px;
}

nav ul li {
  display: inline;
}
```

```
nav ul li a {
    text-decoration: none;
    color: #fff;
    font-weight: bold;
}

nav ul li a:hover {
    color: #ff9800;
}

/* Sections */
section {
    padding: 20px;
    max-width: 800px;
    margin: auto;
}

section h2 {
    color: #444;
}

/* Table */
table {
    width: 100%;
    border-collapse: collapse;
    margin-top: 15px;
}

table, th, td {
    border: 1px solid #555;
    padding: 10px;
    text-align: center;
}

th {
    background-color: #555;
    color: #fff;
}
```

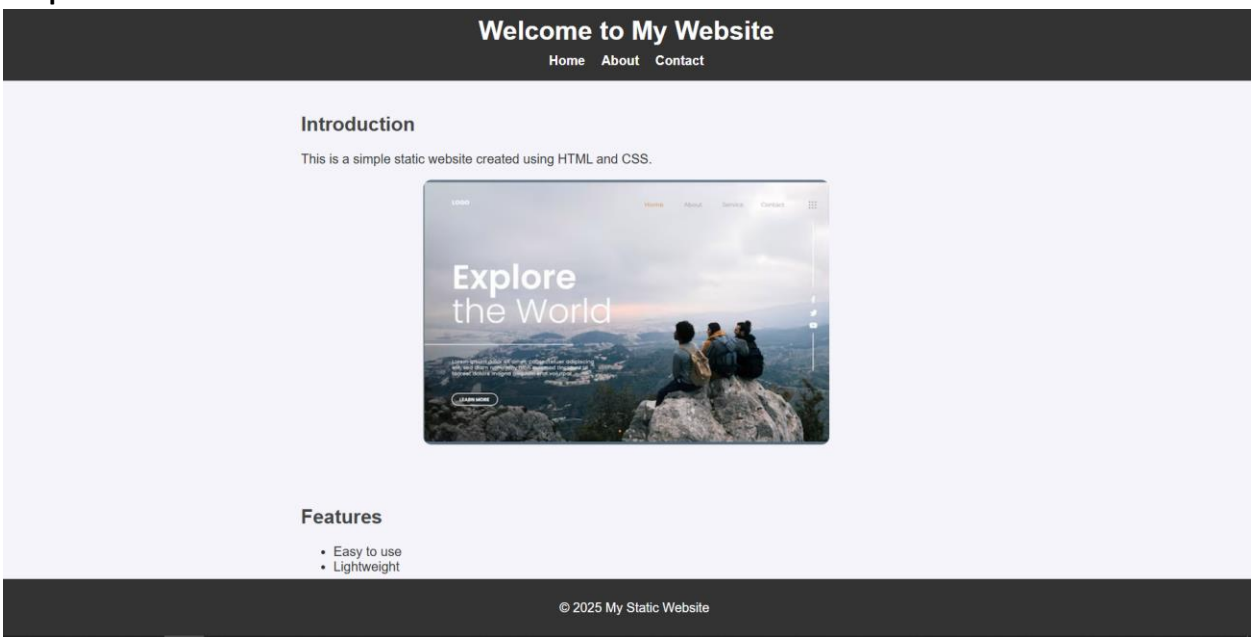
```
/* Image */
img {
   display: block;
   margin: 15px auto;
   border-radius: 10px;
   max-width: 100%;
   hight:580px;
}

/* Footer */
footer {
   background-color: #333;
   color: #fff;
   text-align: center;
   padding: 10px;
   position: fixed;
   width: 100%;
   bottom: 0;
}
```

Expected Output

- A static web page with:
    - A welcome heading.
    - A short paragraph introduction.
    - A navigation menu with links.
    - An “About Me” section.
    - An ordered list of hobbies.
- CSS applied with background colors, styled text, and hover effects for links.

Output:



Conclusion: Successfully created a static web page using HTML for structure and CSS for styling.

Viva/Oral Questions

1. What is the difference between HTML and CSS?
2. What are the three types of CSS?
3. Explain the difference between an ordered list and an unordered list.
4. What is the purpose of the <head> tag in HTML?
5. How does an external CSS file improve website maintenance?

Experiment 2: Interactive Forms with JavaScript Validation

Objective: To create a web form using HTML., To validate form inputs using JavaScript before submission.

Theory:

Web Forms in HTML

- **Purpose:** Web forms are essential for **collecting user input** and interacting with web applications.
- **Elements Used:**
  - <input> → For text, password, email, number, etc.
  - <textarea> → For multi-line input (e.g., comments, feedback).
  - <select> → For dropdown menus.
  - <button> or <input type="submit"> → To submit the form.
- **Examples of Applications:**
  - Login forms, Registration forms, Search boxes, Feedback forms, Payment gateways.
- **Limitation:** HTML forms only **collect input**; they do not check if the input is valid. This is where JavaScript is needed.

JavaScript Validation

JavaScript provides **client-side validation** before form data is submitted to the server. This ensures data is accurate and complete, saving server resources and improving user experience.

Types of Validation

1. **Required Field Validation**
   - Ensures all mandatory fields are filled.
   - Example: A login form must have both username and password entered.
2. **Format Validation**
   - Checks if the input matches a specific pattern.
   - Example: Email should follow user@example.com format.
   - Example: Phone number should contain only digits.
3. **Range/Length Validation**
   - Ensures values are within acceptable ranges.
   - Example: Password length ≥ 6 characters.
   - Example: Age must be between 18 and 60.

Advantages of Client-side Validation (JavaScript)

- Reduces **server load** by catching errors early.
- Provides **instant feedback** to the user.
- Improves **user experience** with real-time checks.

Procedure

1. Create a folder named exp2.
2. Inside it, create form.html and form.js.
3. Design a form with fields like Name, Email, Password, and Phone Number in form.html.
4. Write JavaScript functions in form.js to validate form inputs.
5. Link form.js in form.html.
6. Test the form by entering correct and incorrect data.

Code:

Form.html

```
<!DOCTYPE html>
<html>
<head>
  <title>Form Validation Example</title>
  <script src="form.js" defer></script>
  <style>
    body {
      font-family: Arial, sans-serif;
      background-color: #f9f9f9;
      margin: 20px;
    }
h1 {
      text-align: center;
      color: #333;
       }
    form {
      max-width: 400px;
      margin: auto;
      background: #fff;
      padding: 20px;
      border: 1px solid #ccc;
      border-radius: 8px;
    }
    label {
      display: block;
      margin-top: 10px;
      font-weight: bold;
    }
  input {
      width: 95%;
      padding: 8px;
      margin-top: 5px;
      border: 1px solid #ccc;
      border-radius: 5px;
    }
.error {
      color: red;
      font-size: 14px;
    }
    button {
      margin-top: 15px;
      padding: 10px;
      width: 100%;
     background-color: #0066cc;
```

```
    color: white;
    border: none;
    border-radius: 5px;
    font-size: 16px;
    cursor: pointer;
   }
   button:hover {
    background-color: #004080;
   }
  </style>
</head>
<body>
  <h1>Registration Form</h1>
  <form onsubmit="return validateForm()">
   <label for="name">Name:</label>
   <input type="text" id="name">
   <p class="error" id="nameError"></p>

   <label for="email">Email:</label>
   <input type="text" id="email">
   <p class="error" id="emailError"></p>

   <label for="password">Password:</label>
   <input type="password" id="password">
   <p class="error" id="passwordError"></p>
   <label for="phone">Phone Number:</label>
   <input type="text" id="phone">
   <p class="error" id="phoneError"></p>
   <button type="submit">Submit</button>
  </form>
</body>
</html>
```

form.js

```
function validateForm() {
  let isValid = true;

  // Get form values
  let name = document.getElementById("name").value.trim();
  let email = document.getElementById("email").value.trim();
  let password = document.getElementById("password").value.trim();
  let phone = document.getElementById("phone").value.trim();

  // Clear previous error messages
  document.getElementById("nameError").innerText = "";
  document.getElementById("emailError").innerText = "";
  document.getElementById("passwordError").innerText = "";
  document.getElementById("phoneError").innerText = "";

  // Name validation
  if (name === "") {
    document.getElementById("nameError").innerText = "Name is required.";
    isValid = false;
  }

  // Email validation (basic regex)
  let emailPattern = /^[^ ]+@[^ ]+\.[a-z]{2,3}$/;
  if (!email.match(emailPattern)) {
    document.getElementById("emailError").innerText = "Enter a valid email.";
    isValid = false;
  }
  // Password validation
  if (password.length < 6) {
    document.getElementById("passwordError").innerText = "Password must be at least 6 characters.";
    isValid = false;
  }

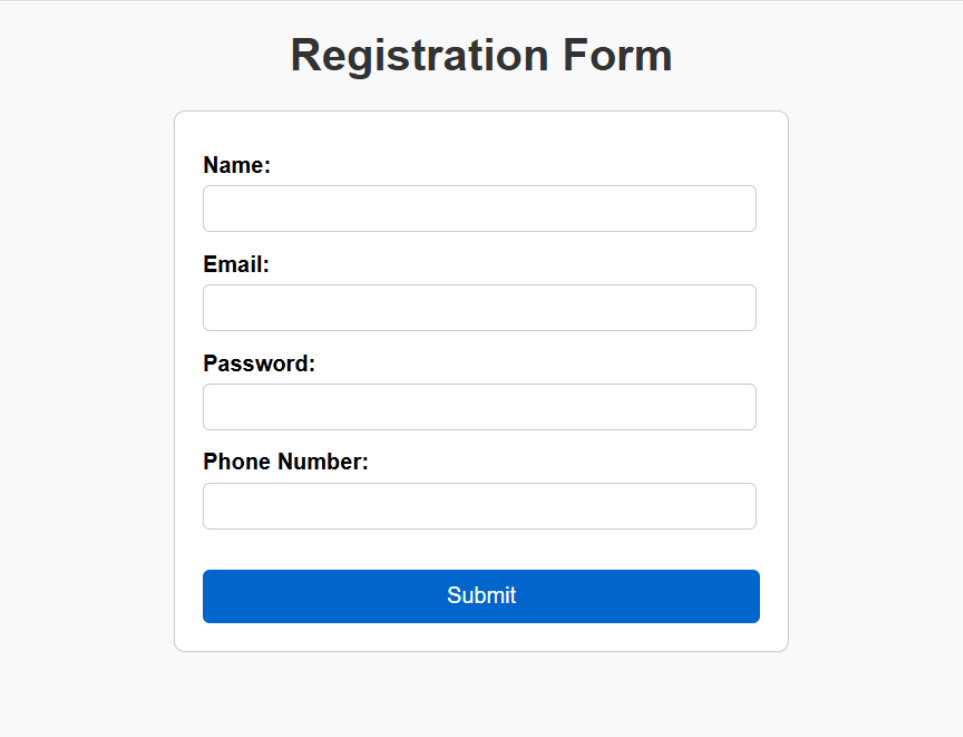
  // Phone validation (10 digits)
  let phonePattern = /^[0-9]{10}$/;
  if (!phone.match(phonePattern)) {
    document.getElementById("phoneError").innerText = "Enter a valid 10-digit phone number.";
    isValid = false;
  }

  return isValid;
}
```

Expected Output

- A **registration form** with fields: Name, Email, Password, Phone Number.
- Error messages are displayed if inputs are invalid (e.g., blank name, incorrect email format, weak password, invalid phone number).
- If all fields are valid, the form will be successfully submitted.

Output: -



Conclusion: Successfully created a form and implemented **JavaScript validation** for user inputs.

Viva/Oral Questions

1. Why is JavaScript used for form validation?
2. What is the difference between client-side validation and server-side validation?
3. Write the JavaScript syntax for an if condition.
4. What happens if form validation is not implemented?
5. What is the difference between onsubmit and onclick events?

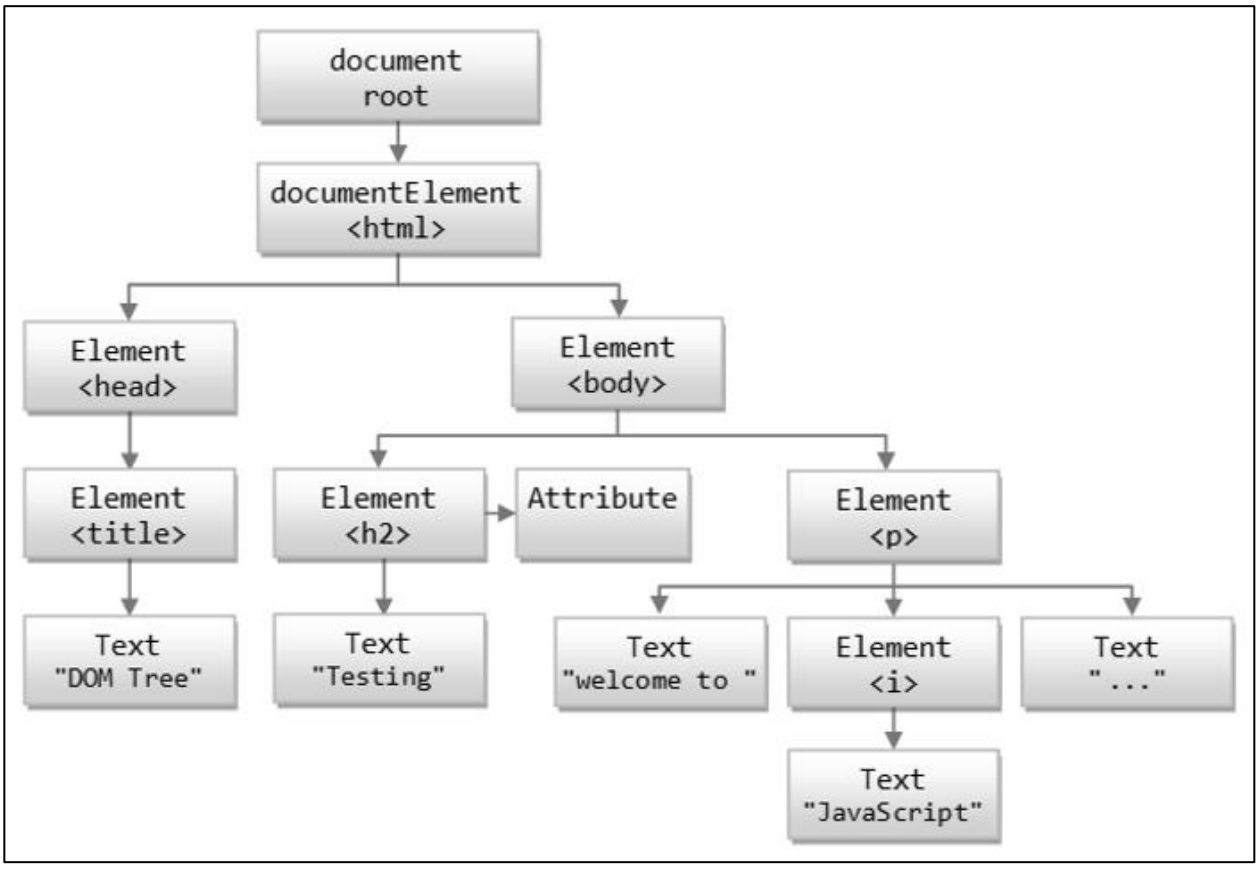
Experiment 3: JavaScript DOM Manipulation and Event Handling

Objective: To learn **Document Object Model (DOM)** manipulation using JavaScript., To dynamically **add, remove, or modify content** in a webpage., To understand and apply **event handling** for interactivity.

Theory

DOM (Document Object Model)

- **Definition:** The DOM is a **programming interface** for HTML and XML documents. It represents a web page as a **tree of nodes**, where each element, attribute, and piece of text is a node that can be accessed and manipulated.
- **Purpose:** Allows JavaScript to dynamically **read, modify, add, or delete content** on a web page without reloading it.



DOM Manipulation Examples

JavaScript can **manipulate the DOM** to make pages dynamic and interactive. Some common operations include:

1. **Adding Elements**
    - Insert new items, like adding <li> to a list dynamically.

- Example:

```
let li = document.createElement("li");
li.textContent = "New Item";
document.getElementById("myList").appendChild(li);
```

2. **Removing Elements**
   - Delete unwanted elements from the page.
   - Example:

```
let li = document.getElementById("item1");
li.parentNode.removeChild(li);
```

3. **Updating Content or Attributes**
   - Change text, HTML content, or attributes dynamically.
   - Example:

```
document.getElementById("title").textContent = "Updated Title";
document.getElementById("link").href = "https://www.example.com";
```

4. **Changing Styles**
   - Modify CSS properties directly via JavaScript.
   - Example:

```
document.getElementById("box").style.backgroundColor = "lightblue";
document.getElementById("box").style.fontSize = "20px";
```

Event Handling in JavaScript

- **Definition:** Events are **actions or occurrences** detected by the browser, such as user interactions or browser-triggered actions.
- **Purpose:** JavaScript allows developers to **listen for these events** and execute code in response, creating dynamic interactivity.

Common Events:

- onclick → Triggered when an element is clicked.
- onmouseover → Triggered when the mouse hovers over an element.
- onchange → Triggered when the value of an input changes.
- addEventListener() → Preferred method for attaching multiple event listeners.

Example:

```
document.getElementById("btn").addEventListener("click", function() {
  alert("Button clicked!");
});
```

Combining DOM Manipulation and Event Handling

- By **listening for events** and **manipulating DOM nodes**, developers can:
  - Dynamically add/remove content (e.g., interactive lists).
  - Change styles and attributes in response to user actions.
  - Create real-time updates without reloading the page (e.g., live search, dynamic counters).

Procedure

1. Create a folder named exp3.
2. Inside it, create dom.html and dom.js.
3. Design a webpage with a section and buttons (Add Item, Remove Item, Hide/Show Section).
4. Write JavaScript code in dom.js to handle button clicks and modify DOM elements.
5. Link dom.js in dom.html.
6. Test the functionality by clicking buttons to add/remove items and toggle visibility.

Code:

dom.html

```
<!DOCTYPE html>
<html>
<head>
 <title>DOM Manipulation Example</title>
 <script src="dom.js" defer></script>
 <style>
  body {
   font-family: Arial, sans-serif;
   margin: 20px;
  }
  h1 {
   text-align: center;
  }
  #content {
   background-color: #f0f0f0;
   padding: 15px;
   border-radius: 8px;
   margin: 20px auto;
   width: 400px;
  }
  ul {
   padding-left: 20px;
  }
  button {
   margin: 5px;
   padding: 8px 12px;
   background-color: #0073e6;
   color: white;
   border: none;
   border-radius: 5px;
```

```
      cursor: pointer;
    }
    button:hover {
      background-color: #005bb5;
    }
  </style>
</head>
<body>
  <h1>DOM Manipulation and Events</h1>

  <div id="content">
    <h3>My List</h3>
    <ul id="myList">
      <li>Item 1</li>
      <li>Item 2</li>
    </ul>
  </div>

  <div style="text-align: center;">
    <button onclick="addItem()">Add Item</button>
    <button onclick="removeItem()">Remove Last Item</button>
    <button onclick="toggleSection()">Hide/Show Section</button>
  </div>
</body>
</html>
```

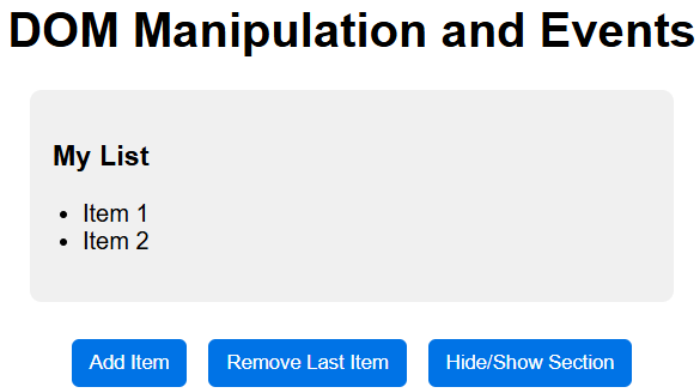
dom.js

```
// Function to add a new list item
function addItem() {
  let ul = document.getElementById("myList");
  let newItem = document.createElement("li");
  newItem.textContent = "New Item " + (ul.children.length + 1);
  ul.appendChild(newItem);
}
// Function to remove the last list item
function removeItem() {
  let ul = document.getElementById("myList");
  if (ul.children.length > 0) {
    ul.removeChild(ul.lastElementChild);
  } else {
    alert("No items left to remove!");
  }
}
// Function to hide or show the content section
function toggleSection() {
  let content = document.getElementById("content");
  if (content.style.display === "none") {
    content.style.display = "block";
  } else {
    content.style.display = "none";
  }
}
```

Expected Output

- Initially, a list with two items is displayed.
- Clicking **Add Item** adds a new list item (Item 3, Item 4, etc.).
- Clicking **Remove Last Item** deletes the last item in the list.
- Clicking **Hide/Show Section** toggles visibility of the entire section.

Output:



Conclusion: Successfully implemented **JavaScript DOM manipulation and event handling** to dynamically add, remove, and toggle webpage content.

Viva/Oral Questions

1. What is the DOM, and why is it important in web development?
2. Differentiate between innerHTML and textContent.
3. What is the difference between onclick and addEventListener()?
4. How can you create and append a new element using JavaScript?
5. Explain the concept of event bubbling in the DOM.

Experiment 4: JavaScript Control Flow and Simple Application

Objective:

- To understand the use of **variables, loops, and conditional statements** in JavaScript.
- To build a **Number Guessing Game** that provides feedback to the user.

Theory:

JavaScript Variables

- Variables are used to **store values** that can be numbers, strings, booleans, arrays, or objects.
- **Declaration Keywords:**
  - var → Function-scoped, can be redeclared (older syntax).
  - let → Block-scoped, modern standard for variables.
  - const → Block-scoped, value cannot be reassigned.

Example:

```
let name = "John";
const maxAttempts = 5;
var score = 0;
```

Conditional Statements

- **Purpose:** Make decisions based on conditions.

Syntax:

```
if (condition) {
  // code executes if condition is true
} else if (anotherCondition) {
  // code executes if previous condition is false but this is true
} else {
  // code executes if all conditions are false
}
```

Loops

- **Purpose:** Execute a block of code repeatedly until a condition is met.
- **Common Loops:**
  - for loop → Executes a block for a known number of iterations.
  - while loop → Executes a block as long as a condition is true.

Example:

```
for (let i = 0; i < 5; i++) {
  console.log("Attempt " + (i+1));
}
```

Random Numbers in JavaScript

- **Math.random():** Generates a random floating-point number between **0 (inclusive) and 1 (exclusive)**.
- To generate a number in a specific range:

```
let secretNumber = Math.floor(Math.random() * 100) + 1; // Random number 1-100
```

Number Guessing Game Concept

A **Number Guessing Game** is a practical application combining variables, conditionals, loops, and random numbers:

1. **Generate a Random Number**
   - Using Math.random(), generate a secret number between a defined range (e.g., 1–100).
2. **Take User Input**
   - Use prompt() or HTML input fields to receive guesses.
3. **Compare Input with Secret Number**
   - Use if…else if…else statements to give feedback:
     - "Too high" → if the guess is greater than the secret number.
     - "Too low" → if the guess is smaller.
     - "Correct" → if the guess matches the secret number.
4. **Repeat Until Correct Guess**
   - Use a while or for loop to allow multiple attempts until the player guesses correctly.

Procedure

1. Create a folder named exp4.
2. Inside it, create game.html and game.js.
3. In game.html, design an input box and button for the user to enter guesses.
4. In game.js, generate a random number between 1 and 100.
5. Use **loops and conditionals** to compare the guess with the random number.
6. Provide feedback messages: "Too high", "Too low", or "Correct".
7. Reload the game once the correct number is guessed.

Code:

game.html

```
<!DOCTYPE html>
<html>
<head>
 <title>Number Guessing Game</title>
 <script src="game.js" defer></script>
 <style>
  body {
   font-family: Arial, sans-serif;
   text-align: center;
   margin: 50px;
   background-color: #f9f9f9;
  }
```

```
    h1 {
      color: #333;
    }
      input, button {
      padding: 10px;
      margin: 10px;
      border-radius: 5px;
      border: 1px solid #ccc;
      font-size: 16px;
    }
    button {
      background-color: #0073e6;
      color: white;
      cursor: pointer;
    }
    button:hover {
      background-color: #005bb5;
    }
    #message {
      margin-top: 20px;
      font-size: 18px;
      font-weight: bold;
    }
  </style>
</head>
<body>
  <h1>Number Guessing Game</h1>
  <p>Guess a number between 1 and 100</p>
   <input type="number" id="guess" placeholder="Enter your guess">
   <button onclick="checkGuess()">Submit Guess</button>
  <p id="message"></p>
</body>
</html>
```

game.js

```
// Generate a random number between 1 and 100
let secretNumber = Math.floor(Math.random() * 100) + 1;
let attempts = 0;

function checkGuess() {
  const guessInput = document.getElementById("guess");
  const message = document.getElementById("message");
  const guess = Number(guessInput.value);

  attempts++;

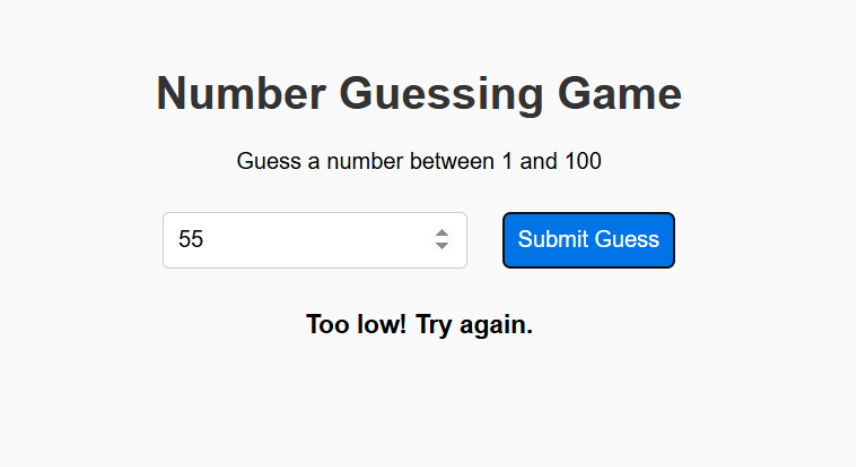
  if (isNaN(guess)) {
    message.innerText = "Please enter a valid number!";
  } else if (guess < secretNumber) {
    message.innerText = "Too low! Try again.";
  } else if (guess > secretNumber) {
    message.innerText = "Too high! Try again.";
  } else {
    message.innerText = "Correct! You guessed it in " + attempts + " attempts.";
    setTimeout(() => {
      alert("Game restarting...");
      location.reload(); // restart game
    }, 2000);
  }
}
```

Expected Output

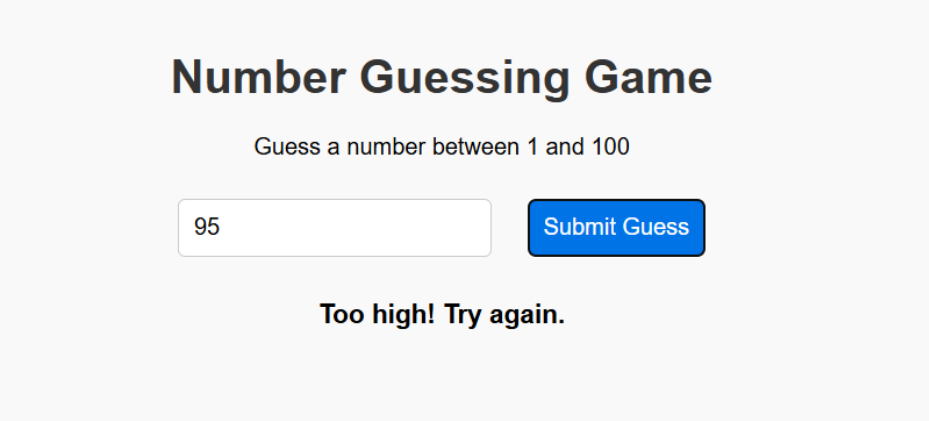
- User enters a number between 1 and 100.
- If the guess is **lower** than the secret number → Output: *"Too low! Try again."*
- If the guess is **higher** than the secret number → Output: *"Too high! Try again."*
- If the guess is **equal** to the secret number → Output: *"Correct! You guessed it in X attempts."*

Output:

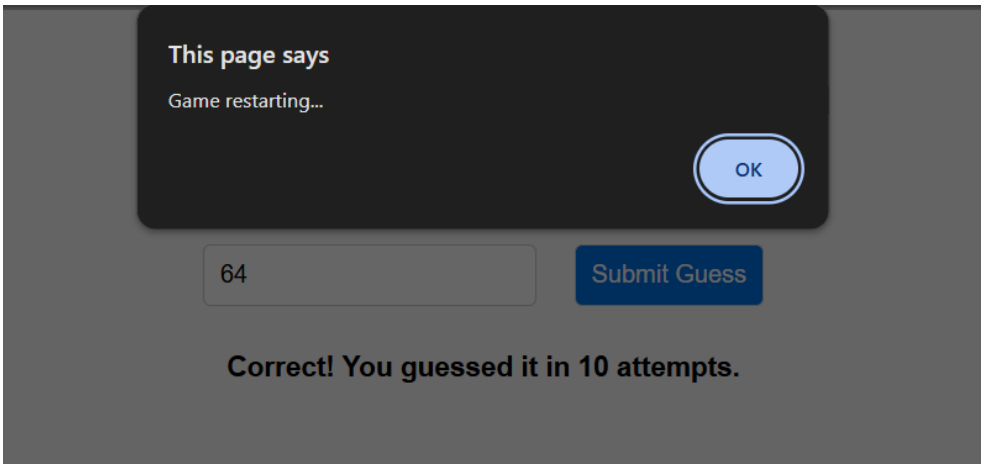
1st attempt



2nd attempt



After 10 Attempts (Correct guessed and Game restarting)



Conclusion: Successfully implemented a **Number Guessing Game** using JavaScript variables, loops, and conditional statements with interactive feedback.

Viva/Oral Questions

1. How do you generate a random number in JavaScript?
2. Differentiate between = = and = = = operators in JavaScript.
3. What is the difference between var, let, and const?
4. Explain the use of **if-else** in JavaScript with an example.
5. How does JavaScript handle user input validation in games or applications?

Experiment 5: PHP Variable Handling and Data Types

Objective:

- To understand PHP variables and data types.
- To declare variables of different data types (string, integer, float, boolean, array).
- To perform simple arithmetic operations in PHP and display results along with data types.

Theory:

PHP Variables

- **Definition:** A variable in PHP is a **container that stores data** which can be used and manipulated in a script.
- **Syntax:** Variable names always start with a **$** symbol, followed by letters or underscores.

```
$name = "John";
$age = 25;
$price = 99.99;
$isStudent = true;
```

- **Rules:**
  - Must start with a $ followed by a letter or underscore.
  - Case-sensitive ($Name and $name are different).
  - No spaces allowed in variable names.

PHP Data Types

PHP supports several common data types:

1. **String**
   - Sequence of characters enclosed in quotes.
   - Example: $greeting = "Hello World";
2. **Integer**
   - Whole numbers without decimals.
   - Example: $age = 25;
3. **Float/Double**
   - Numbers with decimal points.
   - Example: $price = 10.5;
4. **Boolean**
   - Logical values: true or false.
   - Example: $isStudent = true;
5. **Array**
   - A collection of multiple values stored in a single variable.
   - Example: $numbers = array(1, 2, 3, 4, 5);
6. **NULL**
   - Represents a variable with no value.
   - Example: $value = NULL;

Type Checking Functions in PHP

- **gettype($var):** Returns the **data type** of the variable.

```
$x = 25;
echo gettype($x); // Output: integer
```

- **var_dump($var):** Displays the **type and value** of the variable.

```
$name = "John";
var_dump($name); // Output: string(4) "John"
```

Arithmetic Operators in PHP

PHP supports standard arithmetic operations for calculations:

Operator	Description	Example	Output
+	Addition	5 + 3	8
-	Subtraction	5 - 3	2
*	Multiplication	5 * 3	15
/	Division	10 / 2	5
%	Modulus (remainder)	10 % 3	1

Example:

```
$price1 = 50;
$price2 = 30;
$total = $price1 + $price2;  // Addition
$discount = $total * 0.1;    // 10% discount
echo "Total: $total, Discount: $discount";
```

Procedure

1. Install and start **XAMPP/WAMP/LAMP** server.
2. Create a folder exp5 inside the htdocs directory.
3. Inside it, create a file named variables.php.
4. Write a PHP script to declare different types of variables and perform arithmetic.
5. Use gettype() and var_dump() to display data types.
6. Open the script in a browser using:

```
http://localhost/exp5/variables.php
```

Code:
variables.php

```
<!DOCTYPE html>
<html>
<head>
  <title>PHP Variable Handling and Data Types</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      margin: 20px;
      background-color: #f9f9f9;
    }
    h1 {
      color: #333;
    }
    p {
      font-size: 16px;
      margin: 5px 0;
    }
    .box {
      background: #fff;
      padding: 15px;
      border: 1px solid #ccc;
      border-radius: 8px;
      max-width: 500px;
    }
  </style>
</head>
<body>
  <h1>PHP Variable Handling and Data Types</h1>
  <div class="box">
    <?php
      // String variable
      $name = "Alice";
      echo "<p>String: $name (" . gettype($name) . ")</p>";

      // Integer variable
      $age = 25;
      echo "<p>Integer: $age (" . gettype($age) . ")</p>";

      // Float variable
      $price = 99.99;
      echo "<p>Float: $price (" . gettype($price) . ")</p>";

      // Boolean variable
      $isStudent = true;
      echo "<p>Boolean: " . ($isStudent ? "true" : "false") . " (" . gettype($isStudent) . ")</p>";
```

```
    // Array variable
    $colors = array("Red", "Green", "Blue");
    echo "<p>Array: ";
    var_dump($colors);
    echo "</p>";

    // Arithmetic operations
    $a = 10;
    $b = 3;
    echo "<h3>Arithmetic Operations</h3>";
     echo "<p>$a + $b = " . ($a + $b) . "</p>";
    echo "<p>$a - $b = " . ($a - $b) . "</p>";
    echo "<p>$a * $b = " . ($a * $b) . "</p>";
    echo "<p>$a / $b = " . ($a / $b) . "</p>";
    echo "<p>$a % $b = " . ($a % $b) . "</p>";
  ?>
 </div>
</body>
</html>
```

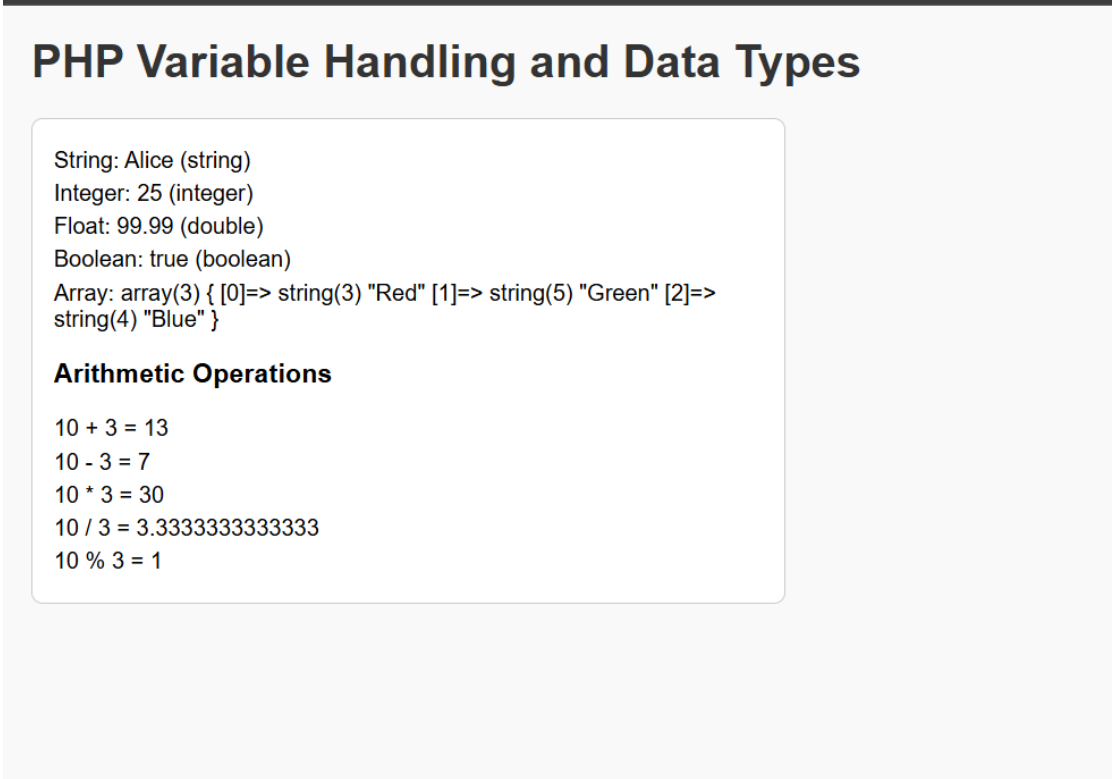
Expected Output

- The script displays variables of **different data types** (string, integer, float, boolean, array).
- The **data type** of each variable is printed using gettype() or var_dump().
- Arithmetic results of addition, subtraction, multiplication, division, and modulus are shown.

```
String: Alice (string)
Integer: 25 (integer)
Float: 99.99 (double)
Boolean: true (boolean)
Array: array(3) { [0]=> string(3) "Red" [1]=> string(5) "Green" [2]=> string(4) "Blue" }

Arithmetic Operations
10 + 3 = 13
10 - 3 = 7
10 * 3 = 30
10 / 3 = 3.3333
10 % 3 = 1
```

Output:



Conclusion: Successfully created a PHP script that demonstrates **variable handling, data types, and arithmetic operations**.

Viva/Oral Questions

1. What are the basic data types in PHP?
2. What is the difference between gettype() and var_dump() in PHP?
3. How are variables declared in PHP?
4. What are the rules for naming variables in PHP?
5. Write a PHP program to swap two numbers using variables.

Experiment 6: PHP Conditional Logic and Looping

Objective

- To understand the use of **conditional statements (if-else)** and **loops (for/while)** in PHP.
- To write a PHP script that:
    1. Displays all **even numbers between 1 and 50**.
    2. Identifies and displays **prime numbers** using conditional logic.

Theory:

1. Conditional Statements in PHP

Conditional statements are used to perform **different actions based on conditions**.

- **if statement** – Executes code if the condition is true.

```
if ($num > 0) {
   echo "Positive number";
}
```

- **if-else statement** – Executes one block if the condition is true, another if false.

```
if ($num % 2 == 0) {
   echo "Even";
} else {
   echo "Odd";
}
```

- **if-elseif-else statement** – Handles multiple conditions.

```
if ($marks >= 75) {
   echo "Distinction";
} elseif ($marks >= 50) {
   echo "Pass";
} else {
   echo "Fail";
}
```

- **switch statement** – Used when a variable is compared against multiple constant values.

```
switch ($day) {
   case 1: echo "Monday"; break;
   case 2: echo "Tuesday"; break;
   default: echo "Invalid Day";
}
```

2. Loops in PHP

Loops are used to execute a block of code repeatedly.

- **for loop** – Used when the number of iterations is known.

```
for ($i = 1; $i <= 5; $i++) {
   echo $i . " ";
}
```

- **while loop** – Runs until a condition is false.

```
$i = 1;
while ($i <= 5) {
   echo $i . " ";
   $i++;
}
```

- **foreach loop** – Special loop for iterating arrays.

```
$colors = array("Red", "Green", "Blue");
foreach ($colors as $c) {
   echo $c . " ";
}
```

3. Even Numbers

- A number is **even** if it is divisible by 2.
- Condition in PHP:

```
if ($num % 2 == 0) {
   echo "$num is Even";
}
```

4. Prime Numbers

- A **prime number** is a natural number greater than 1 with only **two divisors**: 1 and itself.
- Examples between **1–50**: 2, 3, 5, 7, 11, 13, …, 47.

Prime Check Logic in PHP:

```
$num = 17;
$isPrime = true;
for ($i = 2; $i <= sqrt($num); $i++) {
   if ($num % $i == 0) {
      $isPrime = false;
      break;
   }
}
if ($isPrime) {
   echo "$num is Prime";
} else {
   echo "$num is Not Prime";
}
```

Procedure

1. Start **XAMPP/WAMP/LAMP** server.
2. Create a folder exp6 inside the htdocs directory.
3. Inside it, create a file named logic_loops.php.
4. Write a PHP script to:
    - Use a loop to display even numbers from 1 to 50.
    - Use if-else condition inside a loop to check and display prime numbers.
5. Run the file in browser:

```
http://localhost/exp6/logic_loops.php
```

Code:

logic_loops.php

```
<!DOCTYPE html>
<html>
<head>
 <title>PHP Conditional Logic and Looping</title>
 <style>
  body {
   font-family: Arial, sans-serif;
   margin: 20px;
   background-color: #f9f9f9;
  }
  h1 {
   color: #333;
  }
  .box {
   background: #fff;
   padding: 15px;
   border: 1px solid #ccc;
   border-radius: 8px;
   margin-bottom: 20px;
   max-width: 600px;
```

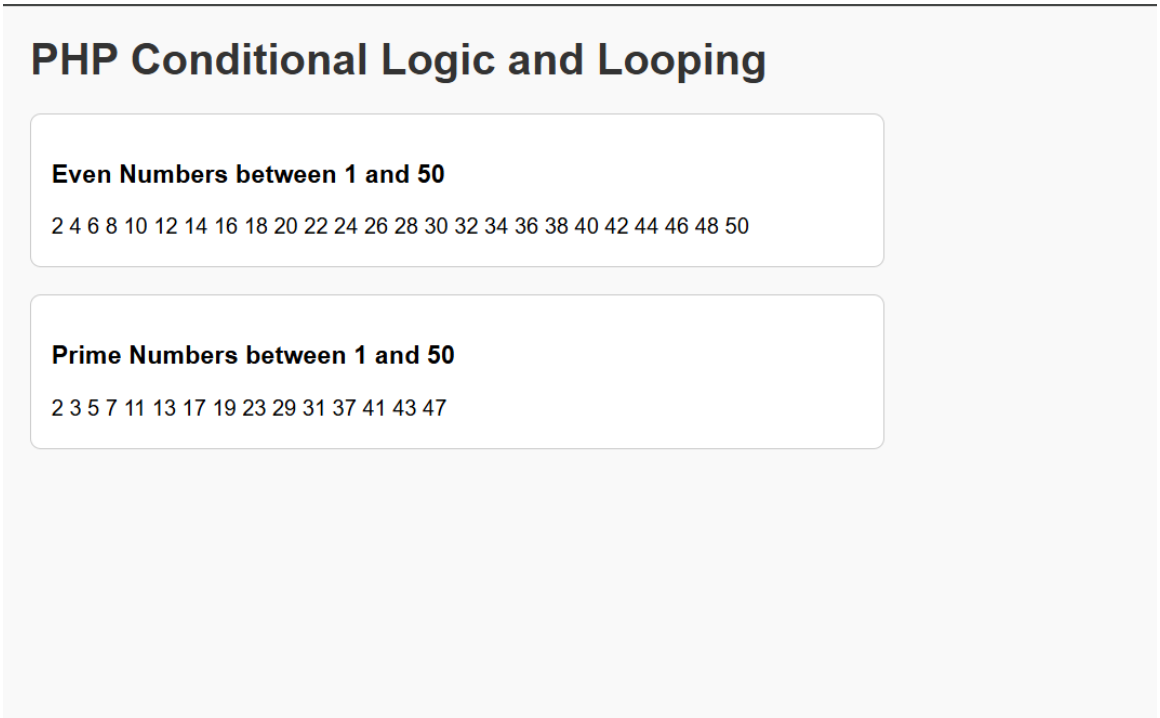
```
    }
p {
      font-size: 16px;
      margin: 5px 0;
    }
  </style>
</head>
<body>
  <h1>PHP Conditional Logic and Looping</h1>
  <div class="box">
    <h3>Even Numbers between 1 and 50</h3>
    <p>
      <?php
        for ($i = 1; $i <= 50; $i++) {
          if ($i % 2 == 0) {
            echo $i . " ";
          }
        }
      ?>
    </p>
  </div>

  <div class="box">
    <h3>Prime Numbers between 1 and 50</h3>
    <p>
      <?php
        for ($num = 2; $num <= 50; $num++) {
          $isPrime = true;
          for ($j = 2; $j <= sqrt($num); $j++) {
            if ($num % $j == 0) {
              $isPrime = false;
              break;
            }
          }
          if ($isPrime) {
            echo $num . " ";
          }
        }
      ?>
    </p>
  </div>
</body>
</html>
```

Expected Output

- **Even numbers between 1 and 50:** 2 4 6 8 10 … 50
- **Prime numbers between 1 and 50:** 2 3 5 7 11 13 17 19 23 29 31 37 41 43 47

Output:



Conclusion: Successfully created a PHP script using **loops** and **if-else conditions** to display even and prime numbers between 1 and 50.

Viva/Oral Questions

1. What are the different types of loops available in PHP?
2. How do you check if a number is even in PHP?
3. What is the logic to determine if a number is prime?
4. Can we use a while loop instead of a for loop here? Why?
5. Write a PHP program to print odd numbers between 1 and 100.

Experiment 7: Session and Cookie Management in PHP

Objective

- To understand how to use **sessions** and **cookies** in PHP.
- To implement a simple **login system** using PHP sessions.
- To set and retrieve a **cookie** that remembers the user's preferred theme (e.g., light or dark mode).

Theory:

1. Sessions in PHP

- A **session** is a way to store information (variables) that can be used across multiple pages.
- Unlike cookies, session data is **stored on the server**, and a unique **Session ID** is assigned to each user.
- This ID is usually stored in a cookie or passed via the URL.
- Sessions are widely used for:
    - **Login systems** (storing username once logged in)
    - **Shopping carts** (remembering selected products)
    - **User activity tracking**

Example – Starting a Session and Storing Data:

```
<?php
session_start();  // Start the session
$_SESSION['username'] = "JohnDoe";  // Store session variable
echo "Welcome, " . $_SESSION['username'];
?>
```

Accessing Session Data on Another Page:

```
<?php
session_start();
echo "Hello, " . $_SESSION['username'];
?>
```

2. Cookies in PHP

- A **cookie** is a small piece of data stored in the **user's browser**.
- Cookies can persist even after the browser is closed (until their expiry date).
- They are commonly used for:
    - **Remembering user preferences** (e.g., dark/light theme)
    - **Language settings**
    - **Auto-login functionality**

Example – Setting a Cookie:

```
<?php
setcookie("theme", "dark", time() + 3600, "/");  // expires in 1 hour
echo "Cookie 'theme' is set!";
?>
```

Accessing a Cookie:

```
<?php
if (isset($_COOKIE['theme'])) {
   echo "Preferred Theme: " . $_COOKIE['theme'];
}
?>
```

3. Difference Between Sessions and Cookies

Feature	Session	Cookie
Storage	Server-side	Client-side (browser)
Security	More secure (data not exposed to user)	Less secure (data visible to user)
Lifetime	Ends when browser closes (unless set)	Can persist after browser closes
Usage	Login systems, cart storage	Preferences, auto-login, themes

Procedure

1. Start **XAMPP/WAMP** server.
2. Create a folder exp7 inside htdocs.
3. Create the following PHP file:
    - index.php
4. Access using browser:

```
http://localhost/exp7/index.php
```

Code:

index.php

```
<?php
session_start();

// --- Handle Login ---
if (isset($_POST['login'])) {
   $username = $_POST['username'];
   $password = $_POST['password'];

   // Demo login check
   if ($username == "admin" && $password == "1234") {
      $_SESSION['username'] = $username;
   } else {
      $error = "Invalid Username or Password!";
   }
}

// --- Handle Logout ---
if (isset($_POST['logout'])) {
   session_destroy();
   setcookie("theme", "", time() - 3600, "/"); // clear theme
   header("Location: " . $_SERVER['PHP_SELF']);
   exit();
}

// --- Handle Theme Preference ---
if (isset($_POST['set_theme'])) {
   $theme = $_POST['theme'];
   setcookie("theme", $theme, time() + (86400 * 7), "/"); // valid for 7 days
   $_COOKIE['theme'] = $theme; // update immediately
   $message = "Theme set to $theme!";
}
```

```
// --- Apply Theme ---
$themeColor = isset($_COOKIE['theme']) ? $_COOKIE['theme'] : "white";
?>
<!DOCTYPE html>
<html>
<head>
  <title>Simple PHP Portal</title>
</head>
<body style="background-color: <?= $themeColor ?>;">

<?php if (!isset($_SESSION['username'])): ?>

  <h2>Login</h2>
  <form method="post">
    Username: <input type="text" name="username" required><br><br>
    Password: <input type="password" name="password" required><br><br>
    <input type="submit" name="login" value="Login">
  </form>
  <?php if (!empty($error)) echo "<p style='color:red;'>$error</p>"; ?>

<?php else: ?>

  <h2>Welcome, <?= $_SESSION['username']; ?>!</h2>
  <p>This is a protected page using PHP Session.</p>

  <h3>Theme Preference</h3>
  <form method="post">
    <select name="theme">
      <option value="white" <?= ($themeColor=="white")?"selected":""; ?>>Light</option>
      <option value="black" <?= ($themeColor=="black")?"selected":""; ?>>Dark</option>
      <option value="lightblue" <?= ($themeColor=="lightblue")?"selected":""; ?>>Blue</option>
    </select>
    <input type="submit" name="set_theme" value="Save Theme">
  </form>
  <?php if (!empty($message)) echo "<p>$message</p>"; ?>

  <form method="post" style="margin-top:15px;">
    <input type="submit" name="logout" value="Logout">
  </form>
<?php endif; ?>

</body>
</html>
```

Expected Output

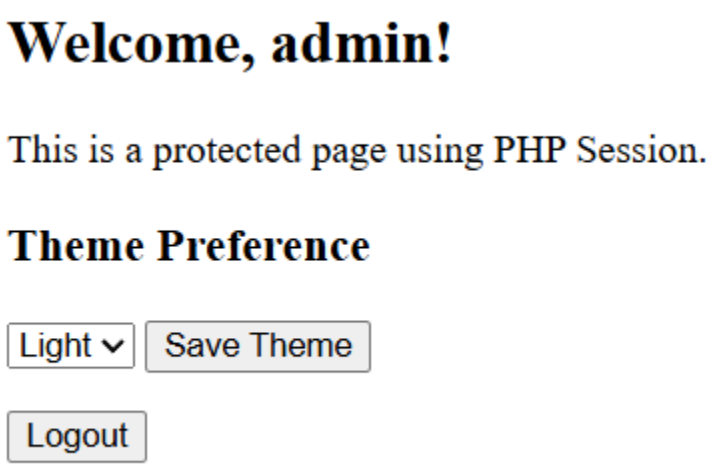
1. **Index Page**: User enters correct credentials (admin / 1234) → "welcome,admin!". then user selects preferred theme → Background color changes and is remembered via cookie. then logout → Ends session and returns to login.

Output:
(Sessions)

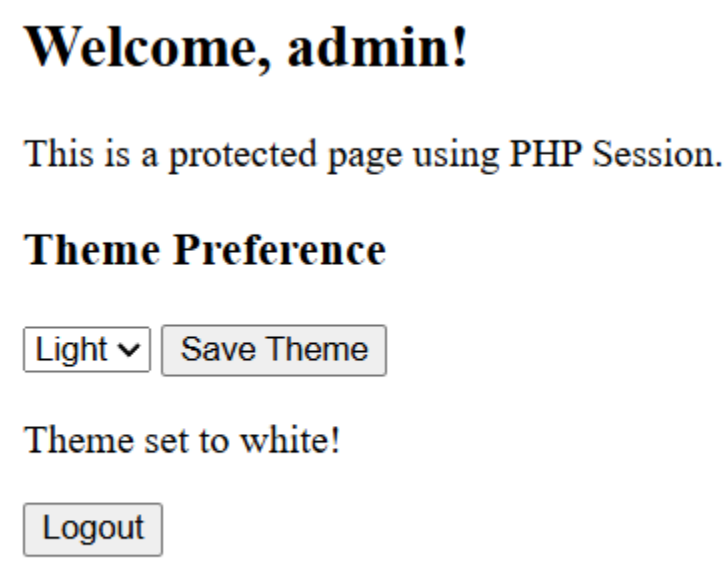
1. Enter Wrong Username and Password



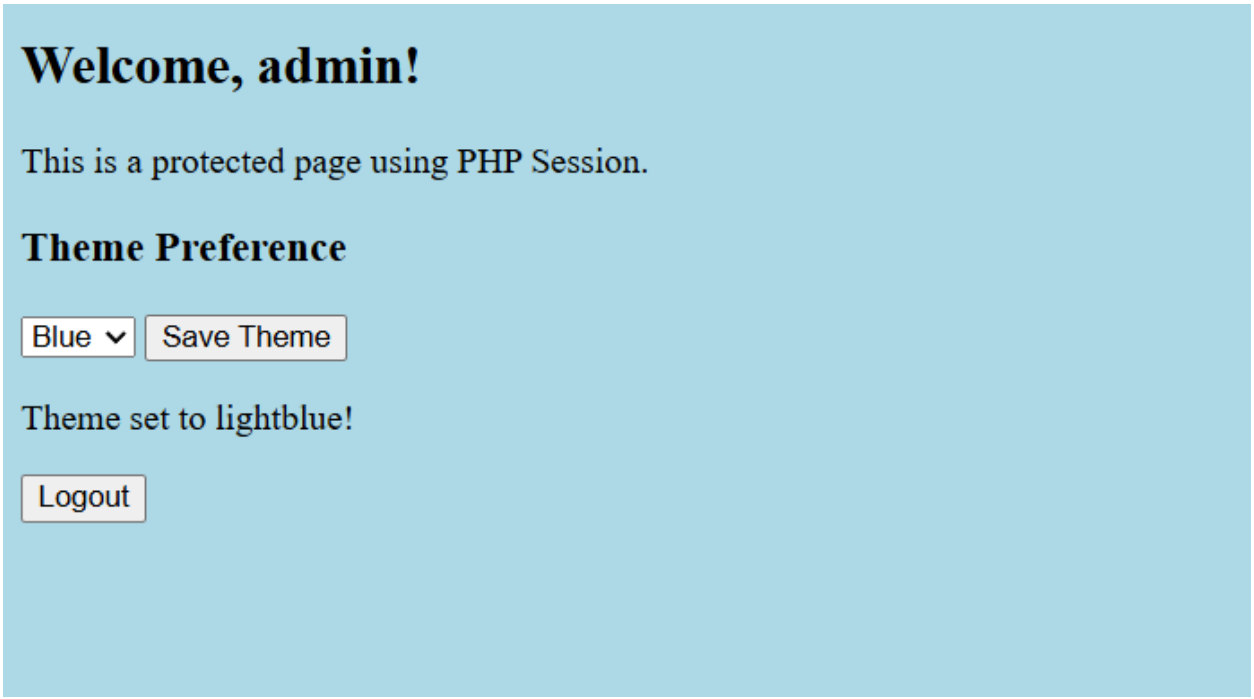
2. Enter correct Username and Password



(Cookies)
Set theme Preference(white)



Set Theme Preference (Lightblue)



Conclusion: Successfully implemented **session management** for login system and **cookie management** for storing user's theme preference in PHP.

Viva/Oral Questions

1. What is the difference between session and cookie?
2. How do you destroy a PHP session?
3. What is the default lifetime of a cookie in PHP?
4. Why are sessions considered more secure than cookies?
5. Write a PHP program to store and display the last login time of a user using cookies.

Experiment 8: Database Connection with PHP

Objective

- To learn how to connect PHP with **MySQL database**.
- To perform **CRUD operations**:
    1. **Create (Insert)** records
    2. **Read (Select)** records
    3. **Update** records
    4. **Delete** records

Theory:

1. Database Connectivity in PHP

- PHP supports database interaction using **MySQLi** (MySQL Improved Extension) and **PDO** (PHP Data Objects).
- To connect with a database, the following details are required:
    - **Hostname** (usually "localhost")
    - **Username** ("root")
    - **Password** (often empty in local setup)
    - **Database Name** (the database you want to use)

Example – Establishing a MySQLi Connection:

```
<?php
$servername = "localhost";
$username = "root";
$password = "";
$database = "student_db";

// Create connection
$conn = new mysqli($servername, $username, $password, $database);

// Check connection
if ($conn->connect_error) {
   die("Connection failed: " . $conn->connect_error);
}
echo "Connected successfully!";
?>
```

2. CRUD Operations in PHP

CRUD stands for **Create, Read, Update, Delete**, the four basic operations used in database applications.

(a) INSERT – Create new records

```
$sql = "INSERT INTO students (name, email) VALUES ('John Doe', 'john@example.com')";
$conn->query($sql);
```

(b) SELECT – Retrieve records

```
$sql = "SELECT id, name, email FROM students";
$result = $conn->query($sql);

while ($row = $result->fetch_assoc()) {
   echo $row["id"]. " - " . $row["name"]. " - " . $row["email"]. "<br>";
}
```

(c) UPDATE – Modify existing records

```
$sql = "UPDATE students SET email='newmail@example.com' WHERE id=1";
$conn->query($sql);
```

(d) DELETE – Remove records

```
$sql = "DELETE FROM students WHERE id=1";
$conn->query($sql);
```

3. Why CRUD is Important

- CRUD is the foundation of **all dynamic web applications**.
- Examples include:
    - **Registration systems** (Insert new users, update profiles)
    - **Content Management Systems (CMS)** (Insert articles, update pages)
    - **E-commerce sites** (Insert new products, delete old stock)

Procedure

1. Start **XAMPP/WAMP** and ensure **Apache + MySQL** services are running.
2. Open **phpMyAdmin** and create a database named exp8db.
3. Inside the database, create a table students with fields:
    - id (INT, Primary Key, Auto Increment)
    - name (VARCHAR(50))
    - email (VARCHAR(100))
    - age (INT)
4. Create a folder exp8 in htdocs.
5. Create the following PHP files:
    - db_connect.php → Database connection file.
    - index.php → Crud operations
6. Run scripts in browser:

```
http://localhost/exp8/index.php
```

Code:

db_connect.php

```
<?php
$servername = "localhost";
$username = "root"; // default in XAMPP
$password = "";
$dbname = "exp8db";

// Create connection
$conn = new mysqli($servername, $username, $password, $dbname);

// Check connection
if ($conn->connect_error) {
   die("Connection failed: " . $conn->connect_error);
}
?>
```

index.php

```
<?php include 'db_connect.php'; ?>
<!DOCTYPE html>
<html>
<head><title>CRUD Operations</title></head>
<body>
 <h2>Insert Student Record</h2>
 <form method="post">
   Name: <input type="text" name="name" required><br><br>
   Email: <input type="email" name="email" required><br><br>
   Age: <input type="number" name="age" required><br><br>
   <input type="submit" name="insert" value="Insert">
 </form>
 <hr>
 <h2>Student Records</h2>
 <table border="1" cellpadding="8">
   <tr>
    <th>ID</th><th>Name</th><th>Email</th><th>Age</th><th>Actions</th>
   </tr>
   <?php
```

```
// --- Insert ---
   if (isset($_POST['insert'])) {
      $name = $_POST['name'];
      $email = $_POST['email'];
      $age = $_POST['age'];

      $sql = "INSERT INTO students (name, email, age) VALUES ('$name','$email','$age')";
      if ($conn->query($sql) === TRUE) {
         echo "<p style='color:green;'>Record inserted successfully</p>";
      } else {
         echo "<p style='color:red;'>Error: " . $conn->error . "</p>";
      }
   }

   // --- Delete ---
   if (isset($_GET['delete'])) {
      $id = $_GET['delete'];
      $conn->query("DELETE FROM students WHERE id=$id");
      echo "<p style='color:red;'>Record deleted successfully</p>";
   }
   // --- Update ---
   if (isset($_POST['update'])) {
      $id = $_POST['id'];
      $name = $_POST['name'];
      $email = $_POST['email'];
      $age = $_POST['age'];
      $conn->query("UPDATE students SET name='$name', email='$email', age='$age' WHERE
id=$id");
      echo "<p style='color:blue;'>Record updated successfully</p>";
   }
   // --- Read / Display Records ---
   $result = $conn->query("SELECT * FROM students");
   while ($row = $result->fetch_assoc()) {
      echo "<tr>
        <td>{$row['id']}</td>
        <td>{$row['name']}</td>
        <td>{$row['email']}</td>
        <td>{$row['age']}</td>
        <td>
          <a href='?delete={$row['id']}' onclick=\"return confirm('Delete this record?')\">Delete</a> |
          <a href='?edit={$row['id']}'>Update</a>
        </td>
      </tr>";
   }
   ?>
  </table>
```

```
 <?php
 // --- Edit Form ---
 if (isset($_GET['edit'])) {
    $id = $_GET['edit'];
    $result = $conn->query("SELECT * FROM students WHERE id=$id");
    $row = $result->fetch_assoc();
 ?>
    <h2>Update Record</h2>
    <form method="post">
      <input type="hidden" name="id" value="<?php echo $row['id']; ?>">
      Name: <input type="text" name="name" value="<?php echo $row['name']; ?>"
required><br><br>
      Email: <input type="email" name="email" value="<?php echo $row['email']; ?>"
required><br><br>
      Age: <input type="number" name="age" value="<?php echo $row['age']; ?>" required><br><br>
      <input type="submit" name="update" value="Update">
    </form>
 <?php } ?>
</body>
</html>
```

Expected Output

1. **Insert:** Enter student details → Record stored in database.
2. **Read/Display:** Displays all student records in tabular form.
3. **Update:** Update student's records.
4. **Delete:** Delete record .

Output:

Insert Student Record

Name:

Email:

Age:

Insert

Student Records

ID	Name	Email	Age	Actions
4	aniket	aniket@gmail.com	18	Delete | Update
5	yogesh	yogesh@gmail.com	25	Delete | Update
6	student	student@123.com	30	Delete | Update

Update Record

Name: yogesh

Email: yogesh@gmail.com

Age: 25

Update

Conclusion: Successfully created PHP scripts to connect with MySQL and performed **CRUD operations** (Insert, Read, Update, Delete).

Viva/Oral Questions

1. What are the two main ways to connect PHP with MySQL?
2. What does CRUD stand for?
3. How do you prevent SQL injection in PHP?
4. What is the difference between mysqli and PDO?
5. Write the SQL query to display students above age 20.

Experiment 9: Display Data from Database using PHP

Objective

- To learn how to **fetch records** from a MySQL database using PHP.
- To **display the data in an HTML table** with proper formatting.
- To apply **basic CSS styling** for better presentation.

Theory:

1. PHP with MySQL Database

- PHP provides **MySQLi** (MySQL Improved) and **PDO** (PHP Data Objects) extensions to connect to a database.
- After establishing a connection, SQL queries can be executed to **fetch, insert, update, or delete records**.
- For displaying data, the most commonly used query is the **SELECT statement**.

2. SELECT Query in PHP

- The **SELECT query** is used to retrieve records from a database table.
- Example SQL Query:

```
SELECT * FROM students;
```

- This retrieves all rows and columns from the `students` table.
- To select specific columns:

```
SELECT name, email FROM students;
```

3. Displaying Data in an HTML Table

- An **HTML table** provides a structured way to display tabular data.
- A table is created using:
  - <table> → Defines the table.
  - <tr> → Defines a row.
  - <td> → Defines a cell (data).
  - <th> → Defines a header cell.

Example (basic table):

```
<table border="1">
 <tr>
  <th>ID</th>
  <th>Name</th>
  <th>Email</th>
 </tr>
 <tr>
  <td>1</td>
  <td>John Doe</td>
  <td>john@example.com</td>
 </tr>
</table>
```

4. CSS Styling for Tables

To make the table more readable and visually appealing, CSS can be applied:

- **Borders:** To separate rows and columns.
- **Padding:** To add spacing inside cells.
- **Background colors:** To highlight headers or alternate rows.
- **Hover effects:** To highlight a row when the mouse hovers over it.

Example CSS:

```
table {
  border-collapse: collapse;
  width: 80%;
  margin: auto;
}
th, td {
  border: 1px solid #333;
  padding: 10px;
  text-align: center;
}
th {
  background-color: #4CAF50;
  color: white;
}
tr:nth-child(even) {
  background-color: #f2f2f2;
}
tr:hover {
  background-color: #ddd;
}
```

Procedure

1. Start **XAMPP/WAMP** server and ensure Apache + MySQL are running.
2. Create a database exp9db (or reuse exp8db).
3. Create a table students with fields: id, name, email, age.
4. Insert sample records into the table using phpMyAdmin.
5. Create a folder exp9 inside htdocs.
6. Inside it, create display.php file.
7. Write PHP code to:
    - Connect to database.
    - Fetch records from students table.
    - Display them inside an HTML table with CSS styling.
8. Run the file in browser:

```
http://localhost/exp9/display.php
```

Code

display.php

```
<?php
// Database Connection
$servername = "localhost";
$username = "root";
$password = "";
$dbname = "exp9db";
$conn = new mysqli($servername, $username, $password, $dbname);
// Check connection
if ($conn->connect_error) {
    die("Connection failed: " . $conn->connect_error);
}

// Fetch records
$sql = "SELECT * FROM students";
$result = $conn->query($sql);
?>
<!DOCTYPE html>
<html>
<head>
  <title>Display Students</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      margin: 20px;
      background: #f4f4f9;
    }
    h2 {
      text-align: center;
      color: #333;
    }
    table {
      width: 80%;
      margin: 20px auto;
      border-collapse: collapse;
      box-shadow: 0 2px 10px rgba(0,0,0,0.1);
      background: #fff;
    }
    th, td {
      border: 1px solid #ddd;
      padding: 12px;
      text-align: center;
    }
```

```
        th {
      background-color: #007BFF;
      color: white;
    }
    tr:nth-child(even) {
      background-color: #f2f2f2;
    }
    tr:hover {
background-color: #d6eaff;
    }
  </style>
</head>
<body>
  <h2>Student Records</h2>
  <table>
    <tr>
      <th>ID</th>
      <th>Name</th>
      <th>Email</th>
      <th>Age</th>
    </tr>
    <?php
    if ($result->num_rows > 0) {
        while($row = $result->fetch_assoc()) {
            echo "<tr>
                    <td>".$row["id"]."</td>
                    <td>".$row["name"]."</td>
                    <td>".$row["email"]."</td>
                    <td>".$row["age"]."</td>
                  </tr>";
        }
    } else {
        echo "<tr><td colspan='4'>No records found</td></tr>";
    }
    ?>
  </table>
</body>
</html>
<?php $conn->close(); ?>
```

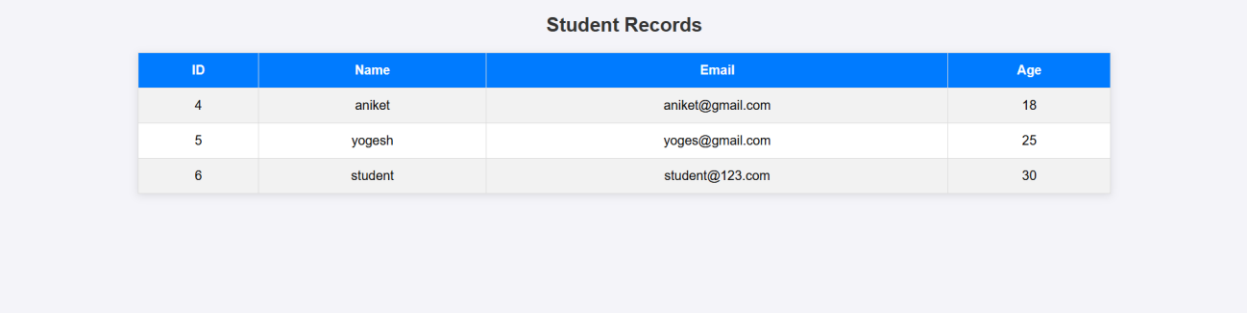
Expected Output

- A styled HTML table displaying all records from the students table.
- Alternate row coloring and hover effects for better readability.

Example Output:

ID	Name	Email	Age
1	John	john@email.com	20
2	Priya	priya@gmail.com	21
3	Ahmed	ahmed@yahoo.com	22

Output:



Conclusion: Successfully created a PHP script to **fetch and display records from MySQL database** in an **HTML table** with CSS styling.

Viva/Oral Questions

1. What is the difference between mysqli_fetch_assoc() and mysqli_fetch_array()?
2. How do you display data in an HTML table using PHP?
3. What is the purpose of while($row = $result->fetch_assoc()) in PHP?
4. Explain the difference between INNER JOIN and LEFT JOIN in SQL.
5. Write an SQL query to display students with age greater than 21.

Experiment 10: Mini Project – Registration System

Objective

- To design and implement a **complete registration system**.
- To integrate **frontend validation (HTML + JavaScript)** with **backend processing (PHP)**.
- To use **MySQL database** for storing user data.
- To implement **CRUD operations**: Create, Read, Update (Edit), and Delete registered users.

Theory:

1. **Registration System Workflow**
    - **Frontend (HTML + CSS + JS):** User fills out the registration form. JavaScript ensures that inputs are valid (e.g., email format, password length).
    - **Backend (PHP):** Form data is submitted to the server and processed by PHP.
    - **Database (MySQL):** User details are stored in a structured table (e.g., users).
    - **CRUD Operations:**
        - **Create** → Insert new user.
        - **Read** → Display list of registered users.
        - **Update** → Allow editing of user details.
        - **Delete** → Remove a user from the system.
2. **Frontend Validation:**
    - Prevents submission of incomplete/invalid forms.
    - Example checks: required fields, valid email format, password match.
3. **Backend Validation:**
    - Ensures data security and correctness.
    - Prevents SQL injection (use prepared statements).
4. **User Management:**
    - Admin can view all registered users.
    - Options for **Edit** and **Delete** next to each record.

Instructions

1. **Database Setup**
    - Create a database registrationdb.
    - Create a table users with fields:
    - id (INT, AUTO_INCREMENT, PRIMARY KEY)
    - name (VARCHAR)
    - email (VARCHAR)
    - password (VARCHAR)
    - phone (VARCHAR)
    - Insert a few sample records for testing.
2. **Frontend (Form Creation)**
    - Create a register.html file with fields: Name, Email, Password, Confirm Password, Phone.
    - Apply **CSS** for layout and styling.
    - Use **JavaScript validation**:

        - Check required fields.
        - Validate email format.
        - Ensure password length ≥ 6 and matches confirm password.
        - Validate phone number (only digits, 10 characters).
3. **Backend (PHP Processing)**
    - Create register.php to process submitted form data.
    - Connect PHP to MySQL using mysqli or PDO.
    - Insert valid records into users table.
    - Redirect to a success page or display a confirmation message.
4. **Display Registered Users**
    - Create users.php to fetch all records from the database and display them in an HTML table.
    - Each row should include **Edit** and **Delete** links.
5. **Edit Functionality**
    - Clicking **Edit** loads user details into a form.
    - Update values in the database using PHP.
6. **Delete Functionality**
    - Clicking **Delete** removes the user's record from the database.
    - Show confirmation before deleting.
7. **Testing**
    - Test with valid and invalid inputs.
    - Test editing and deleting records.
    - Ensure the application works end-to-end.

Expected Outcome

- A fully functional **mini registration system** where users can:
    - Register with frontend + backend validation.
    - View a list of all registered users.
    - Edit or delete user records.